COMPUTACION

Clase 1

Temas de la clase 1:

- ☞ **Creación de Algoritmos** ▶
- ☞ **Introducción al lenguaje Pascal**
  - **Algoritmo programa**
  - **Acciones=sentencias ejecutables**
  - **Estructura de un programa Pascal**
  - **Tipos de datos**
  - **Sentencias de Asignación**
  - **Sentencias de Entrada /Salida**
  - **Prueba de escritorio**

Algoritmo:

Es un procedimiento con las siguientes características:

- **Posee una secuencia finita y ordenada de acciones primitivas (órdenes elementales)**
- **Las acciones primitivas son no ambiguas**
- **La ejecución de la secuencia de acciones primitivas, en el orden dado, conduce a la solución del problema en un tiempo finito**

Problemas

- Sumar números enteros
- Encontrar las raíces de una ecuación cuadrática
- Invertir una matriz de 1000 x 1000 reales
- Resolver un sistema de ecuaciones diferenciales
- Traducir un texto
- Desarrollar un juego de computadora

COMO se resuelven estos problemas en una computadora?

Problema

Encontrar una raíz de la siguiente ecuación cuadrática :

$$x^2 - 4x + 4 = 0$$

Si consideramos la ecuación

$$ax^2 + bx + c = 0$$

Tenemos $a = 1$ $b = -4$ $c = 4$

SOLUCIÓN

El algoritmo tiene nombre!!!!

ALGORITMO RAIZ

1	-4	4

1. Determinar los valores a, b y c, coeficientes de la ecuación
2. Calcular el discriminante
3. Calcular una raíz
4. Fin de algoritmo RAIZ

1. Ingresar a, b, c
2. Discriminante $\sqrt{b^2 - 4*a*c}$
3. Raiz $\dfrac{b + \text{Discriminante}}{2*a}$
4. Fin

Variables:
a, b, c, Raiz, Discriminante

Contexto en el que se diseña un algoritmo:

- PROCESADOR: es toda entidad capaz de “entender” ordenes elementales y ejecutarlas.
- ACCIONES PRIMITIVAS: órdenes elementales entendidas por el procesador que se emplea.
- ACCIONES NO PRIMITIVAS: son órdenes no elementales de una abstracción mayor a las órdenes elementales. Cada una de ellas puede ser dividida en otras acciones (primitivas o no).

Problema

Dados 3 números enteros (cualesquiera), se desea detectar los que son

positivos y escribirlos.

ALGORITMO DECISION

Solución

1. Determinar **a,b,c**
2. Si **a** es positivo entonces escribir **a**
3. Si **b** es positivo entonces escribir **b**
4. Si **c** es positivo entonces escribir **c**
5. Fin del algoritmo DECISION

1. Ingresar **a,b,c**
2. Si **a>0** entonces escribir **a**
3. Si **b>0** entonces escribir **b**
4. Si **c>0** entonces escribir **c**
5. Fin

Variables:
a, b, c

Problema

Dada una cantidad variable de números enteros (cualesquiera), se desea escribir aquellos que sean positivos y aquellos que no lo sean, aclarando cada caso

Solución

ALGORITMO REPETICIÓN

1. Ingresar **cantidad**
2. Repetir desde 1 hasta **cantidad** las siguientes acciones
    3. Ingresar **a**
    4. Si **a>0** entonces escribir **a,**'es positivo'
       sino escribir **a**,'es negativo o cero'
5. Terminar la repetición
6. Fin del algoritmo REPETICIÓN

Variables:
cantidad, a

Ejecución de un algoritmo

La **ejecución** de un algoritmo es una secuencia operaciones calculadas a partir de las acciones primitivas del algoritmo.

La **ejecución** de un algoritmo para una entrada específica de datos constituye la resolución de un caso particular del problema. Esos datos son propios del problema.

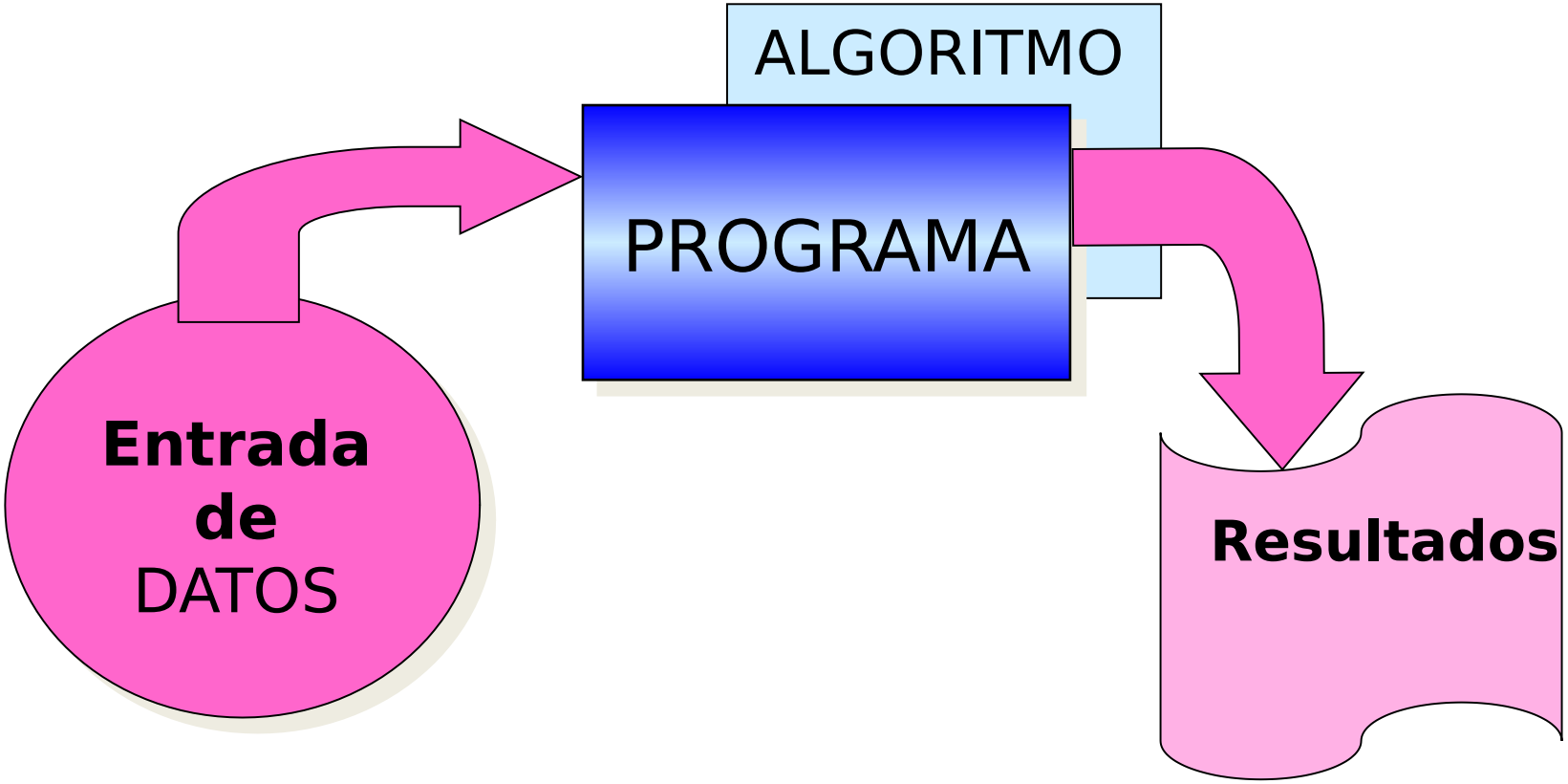
Ejecución de un algoritmo

- Ingresar **cantidad**
- Repetir desde 1 hasta **cantidad** las siguientes acciones
  - Ingresar **a**
  - Si **a>0** entonces escribir **a,**'es positivo'
    sino escribir **a**,'es negativo o cero'

1. Terminar la repetición
2. Fin del algoritmo REPETICIÓN

```
ENTRADA: cantidad ←  4  {Ingresar cantidad}

contador ← 1
a ← 6
6 es positivo {porque 6 es mayor que cero}

contador ← 2
a ← -9
-9 es negativo o cero {porque -9 es menor que cero}

contador ← 3
a ← 0
0 es negativo o cero {porque 0 es cero}

contador ← 4
a ← 2
2 es positivo {porque 2 es mayor que cero}
```

Ciclo elemental



Estructura de un Programa Pascal

PROGRAM Identificador → **Encabezamiento**

USES
Identificador/es
LABEL
Declaración de etiquetas
CONST
Definición de constantes
TYPE
Declaración de tipos
VAR
Declaración de variables
PROCEDURE
Declaración de procedimientos
FUNCTION
Declaración de funciones
BEGIN
Sentencias ejecutables
END.

Tipo de Datos

→ **Bloque**

TIPOS DE DATOS

- Simples
  - Estandar
    - Numérico
      - Entero (integer)
      - Real (real)
    - Carácter (char)
    - Cadena (string)
    - Lógico (boolean)
  - Definidos por el programador
    - Subrango
    - Enumeración
- Estructurados
  - Arreglos (array)
  - Registros (record)
  - Conjuntos (set)
  - Archivos (file)
- Puntero

Un tipo de dato define:

- el conjunto de valores (que puede tomar una variable)
- y las operaciones (que pueden realizarse con ella)

Datos Simples estándar (Free Pascal)

- Numérico
  - Entero (*) (**INTEGER**) [**LONGINT/BYTE/SHORTINT/WORD/CARDINAL/INT64**]
  - Real (**REAL**) [**SINGLE/DOUBLE/EXTENDED**]

- Lógico(*) → **BOOLEAN** – Dos valores: **TRUE** /**FALSE**

- Carácter(*) → **CHAR** - Un único carácter – Comillas simples

(*) Ordinales

Tipo entero (integer)

Tipos	Rango numérico	Bytes (Bits)
Byte	0..255	1 (8)
Shortint	-128..127	1 (8)
Smallint	-32768..32767	2(16)
Integer	-32768..32767	2 (16)
Longint	-2147483648..2147483647	4 (32)
Int64	-9223372036854775808..9223372036854775807	8 (64)
Word	0..65535	2 (16)
Longword	0..4294967295	4(32)
QWord	0 .. 18446744073709551615	8(64)
Cardinal	0..4294967295	4(32)

Tipo real (real)

Tipos	Rango real	Decimales significativos	Bytes (Bits)
Single	1.5^{-45} .. 3.4^{38}	7-8	4 (32)
Real	2.9^{-39} .. 1.7^{38}	15-16	8 (64)
Double	5.0^{-324} .. 1.7^{308}	15-16	8 (64)
Extended	3.6^{-4951} .. 1.1^{4932}	17-18	10 (80)
Comp	$-2^{63}+1$ .. $2^{63}-1$	15-16	8 (64)

Tabla ASCII - caracteres

ACII	0	1	2	3	4	5	6	7	8	9
30	-		blanco	¡	“	#	$	%	&	‘
40	(	)	*	+	,	-	.	/	0	1
50	2	3	4	5	6	7	8	9	:	;
60	<	=	>	¿	@	A	B	C	D	E
70	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y
90	Z	[	\	]	^	_	`	a	b	c
100	d	e	f	g	h	i	j	k	l	m
110	n	o	p	q	r	s	t	u	v	w
120	x	y	z	{	|	}	~		Ç	ü

Como se ve en la tabla, existe un orden entre los caracteres y es posible compararlos en forma aislada **‘A' < ‘h'**

y tambien una secuencia de caracteres (cadena) **‘Papa' > ‘PAPA'**

Notar que es diferente la representación del valor entero 5 (internamente un 5 en binario) del carácter ‘5' (internamente un 53 en binario)

Constantes

Sintaxis

CONST

Identificador de constante = valor de la constante;

Ejemplos:

```
CONST
   Anio=2013;
   Altura=1.68;
   Siglo=21;
```

Declaración de variables
Sintaxis

VAR

Identificador de variable : identificador de tipo;

Ejemplos:

```
VAR
dividendo, divisor, cociente:  integer;
resultado: real;
sigue: boolean;
Nombre:string;
```

- La declaración de variables es obligatoria.
- Cada variable declarada, tiene un espacio propio en la memoria.

Codificar

Acciones del algoritmo → **Sentencias de programa**

<u>Sentencias simples</u>

- Asignación
- Lectura (entrada de datos)
- Escritura (escritura de resultados)

Sentencias Ejecutables

Simples

Ninguna de sus partes puede a su vez considerarse como otra sentencia.

Estructuradas (o Compuestas)

son construcciones compuestas de otras sentencias que deben ser ejecutadas en secuencia (enunciados compuestos), en forma condicional (enunciados condicionales) o en forma reiterada (enunciados de repetición).

Sentencia de asignación

Establece un nuevo valor en una variable, si ésta tuviera un valor previo lo pierde.
El símbolo se traduce como :=

Sintaxis Pascal:

```
Identificador de variable:=expresión;
```

```
Radio:= 5;
Perimetro:= 2*Radio*3.1416;
Superficie:= Radio*Radio* 3.1416;     {78.54}
Radio:=Radio + 5;
Diametro:= 2*Radio;
```

Un contador es una variable de algún tipo entero que cuando se ejecuta se incrementa en una unidad o en una cantidad constante.

```
Contador:=6;
Contador:=Contador+1;
```

Un acumulador es una variable que se incrementa en una cantidad variable.

```
Acum:=12.32;
S:=7.5;
Acum:=Acum + S;
```

Sentencia de entrada

Establece los nuevo/s valor/es que ingresan desde un medio externo (teclado/archivo) en variable/s, si ésta/s tuviera/n un valor previo lo pierde/n.

Se utiliza Read/Readln en vez de **Ingresar.**

Sintaxis

```
Read(v1,v2,...,vn);
Readln(v1,v2,..vn);
```

Ejemplos:

```
Readln(Radio);
Readln(a, b, c);
Read( Archivo, TE, DNI );
```

Por defecto los datos ingresan desde el teclado, para ingresar desde un archivo debe explicitarse en el 1er. parámetro

Sentencia de salida

Escribe en pantalla/archivo la “lista de salida”
Compuesta por variables, constantes o expresiones. Se utiliza *write* o *writeln*

Sintaxis:

```
write(e1,e2,...,en);
writeln(e1,e2,..,en);
```

Ejemplos:

```
write(Lado);
write(‘Hoy comienza el otoño' );
writeln('La superficie es =', Lado*4);
writeln( Archivo, , a , b , c );
```

Por defecto los resultados se visualizan en pantalla, para grabarlos en un archivo debe explicitarse en el 1er. parámetro

```
{visualización de resultados, uso de FORMATO}
Var
   X: real;
   N: integer;
……————————————
X:=3.456
N:=10;
Writeln(X);
Writeln(N);
……————————————
```

```
0.345600 E01
10
```

Es posible darle formato indicando la cantidad de columnas (ancho de campo). Para los valores reales se especifica además la cantidad de decimales.

```
Writeln(X:7:3);
Writeln(N:5);
……————————————
```

```
_ _3.456
_ _ _ 10
```

```
program SuEdad;
Const
  Periodo=5;
Var
  Nombre : string[25];
  AnioNac, AnioAct:integer;
  Edad:byte;
begin
writeln('Ingrese su nombre');
readln(Nombre);
writeln('Ingrese el anio de nacimiento y el actual');
readln(AnioNac, AnioAct);
Edad:= AnioAct - AnioNac;
writeln(Nombre, '  tiene ', Edad ,' anios ');
Edad:= Edad + Periodo;
writeln('dentro de ', Periodo, ' anios tendra ',Edad);
end.
```

No olvidar ; Begin..End.

Vamos a verificar el funcionamiento del programa realizando una prueba de escritorio

¿Qué es una prueba de escritorio?

Prueba de escritorio

- Una prueba de escritorio es la ejecución ‘‘manual'' de un programa a partir de ingreso de datos específicos.

- Se implementa e identifica una columna por cada variable/expresión lógica y se escriben los valores que van tomando las mismas a medida que se ejecutan las sentencias del programa en secuencia desde el BEGIN hasta el END.

- Se presume que el programa es correcto si las variables del resultado contienen los valores esperados. Es muy conveniente hacer varias corridas con diversos juegos de datos a fin de verificar que SIEMPRE da la respuesta correcta

Prueba de escritorio:

```
Readln (Nombre);
Readln(AnioNac, AnioAct);
Edad:= AnioAct - AnioNac;
Writeln(Nombre, 'tiene', Edad, 'anios');
Edad:= Edad + Periodo;
Writeln('Dentro de',Periodo, 'anios tendra', Edad);
```

Lote de Prueba: Maria , 1995, 2015

Nombre	AnioNac	Anio Act	Edad	SALIDA
Maria				
	1995	2015	20	
				Maria tiene 20 anios
			25	
				Dentro de 5 anios tendra 25

Observar que la constante *Periodo* no se representa en una columna, ¿por qué?